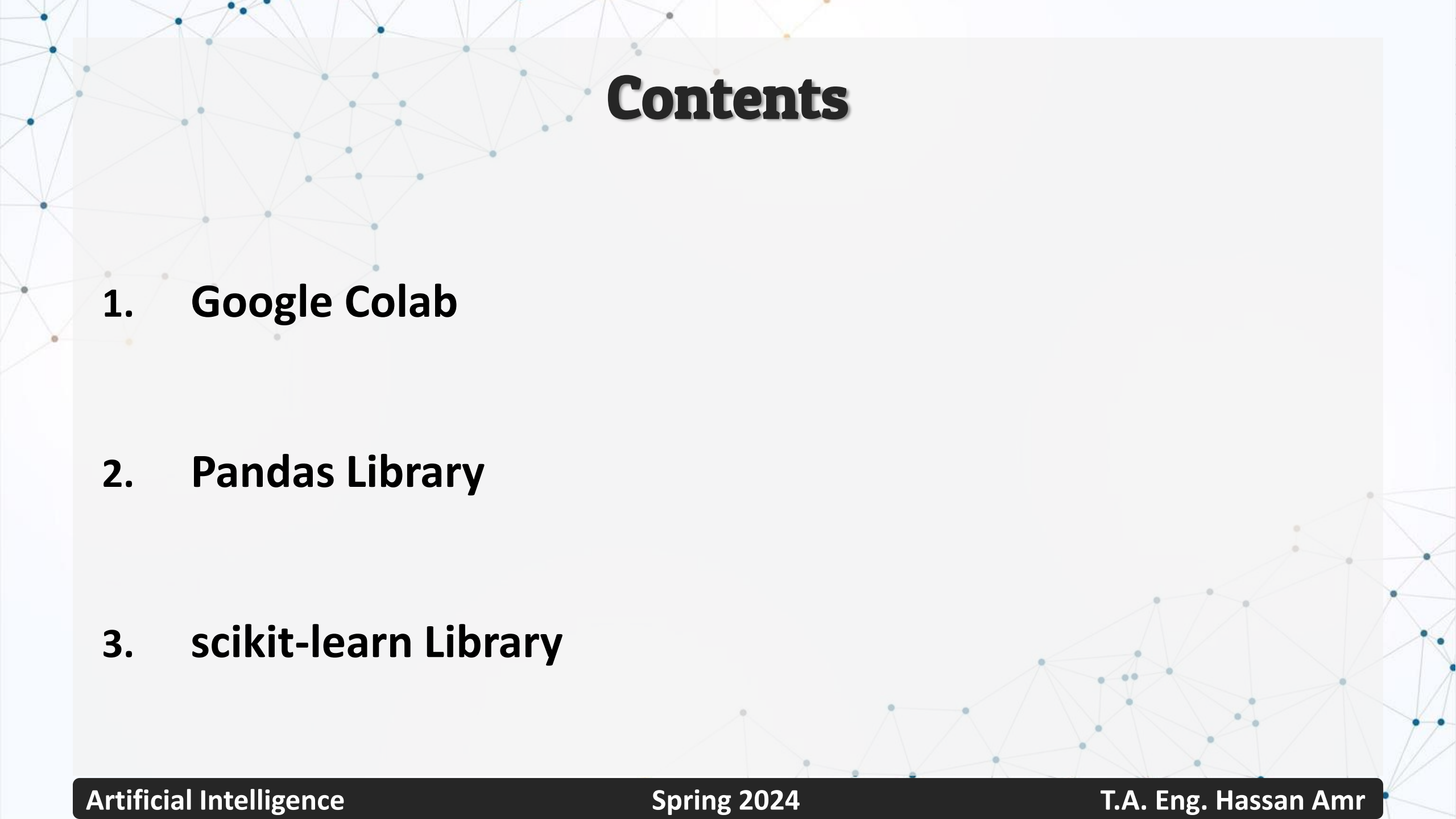


Artificial Intelligence (Spring 2024)

AI Week 2

[Google Colab – Pandas Library – scikit-learn Library]

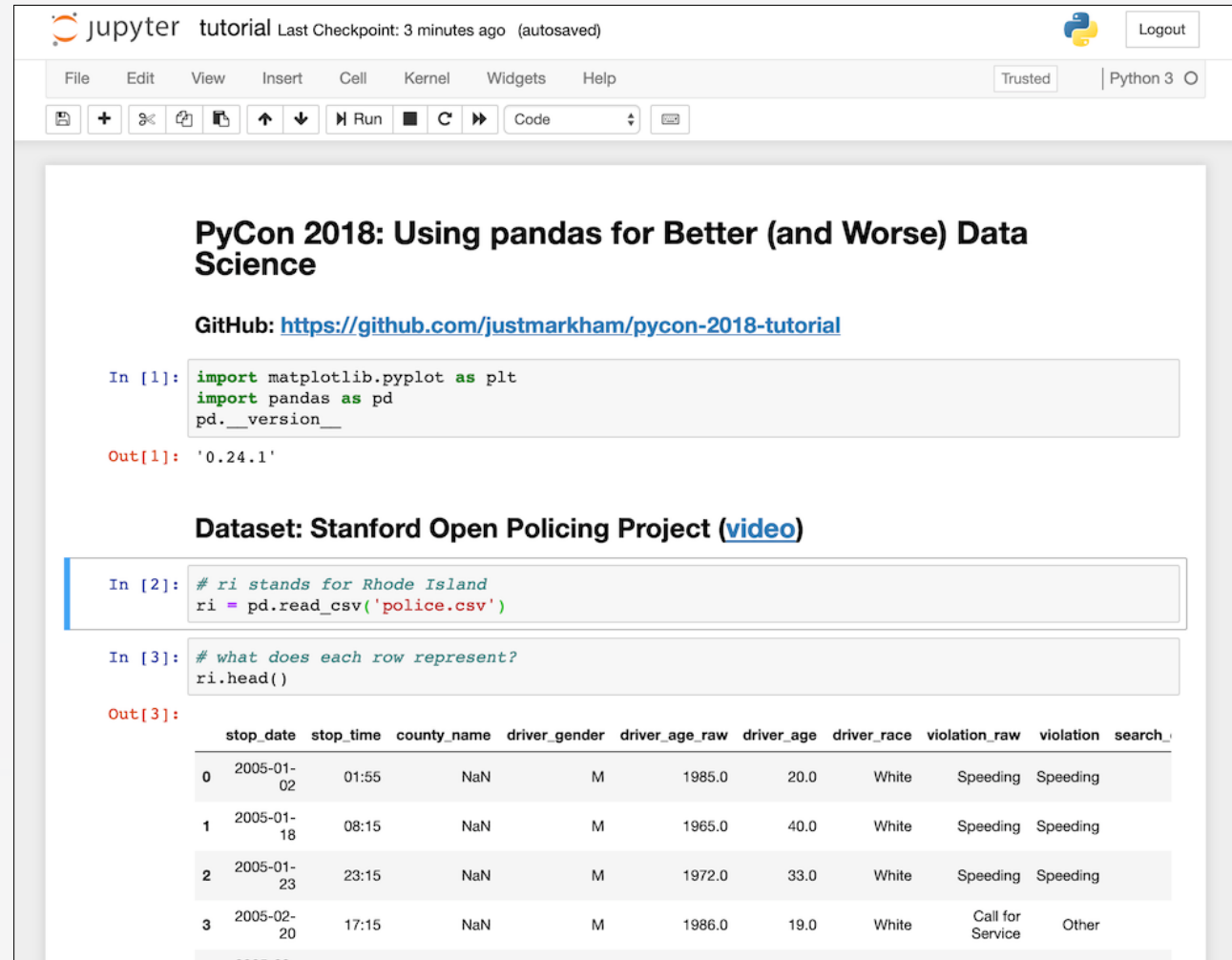


Contents

- 1. Google Colab**
- 2. Pandas Library**
- 3. scikit-learn Library**

Pre-Google Colab (Jupyter Notebook)

The Jupyter Notebook is the original web application for creating and sharing computational documents. It offers a simple, streamlined, document-centric experience.

A screenshot of a Jupyter Notebook interface. The top bar shows 'jupyter tutorial' and 'Last Checkpoint: 3 minutes ago (autosaved)'. The menu bar includes File, Edit, View, Insert, Cell, Kernel, Widgets, and Help. The toolbar has icons for file operations, running, and code execution. The notebook content includes a title 'PyCon 2018: Using pandas for Better (and Worse) Data Science', a GitHub link, a code cell with imports for matplotlib and pandas, an output showing the pandas version, a dataset description 'Stanford Open Policing Project', and a code cell for loading a CSV file. The output of the second code cell is a table of data.

```
In [1]: import matplotlib.pyplot as plt
import pandas as pd
pd.__version__

Out[1]: '0.24.1'
```

Dataset: Stanford Open Policing Project ([video](#))

```
In [2]: # ri stands for Rhode Island
ri = pd.read_csv('police.csv')

In [3]: # what does each row represent?
ri.head()
```

Out[3]:

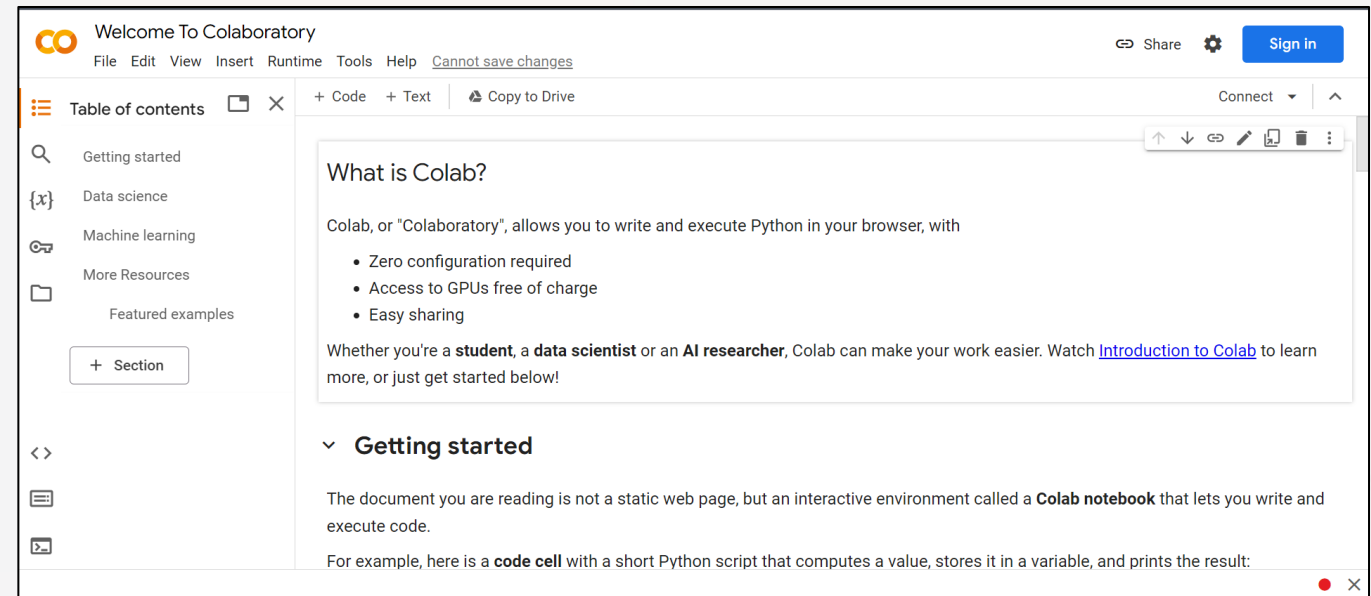
	stop_date	stop_time	county_name	driver_gender	driver_age_raw	driver_age	driver_race	violation_raw	violation	search_
0	2005-01-02	01:55	NaN	M	1985.0	20.0	White	Speeding	Speeding	
1	2005-01-18	08:15	NaN	M	1965.0	40.0	White	Speeding	Speeding	
2	2005-01-23	23:15	NaN	M	1972.0	33.0	White	Speeding	Speeding	
3	2005-02-20	17:15	NaN	M	1986.0	19.0	White	Call for Service	Other	

Google Colab



Colab is a free Jupyter notebook environment that runs entirely in the cloud.

It does not require a setup and the notebooks that you create can be simultaneously edited by your team members - just the way you edit documents in Google Docs.



Why Google Colab ?

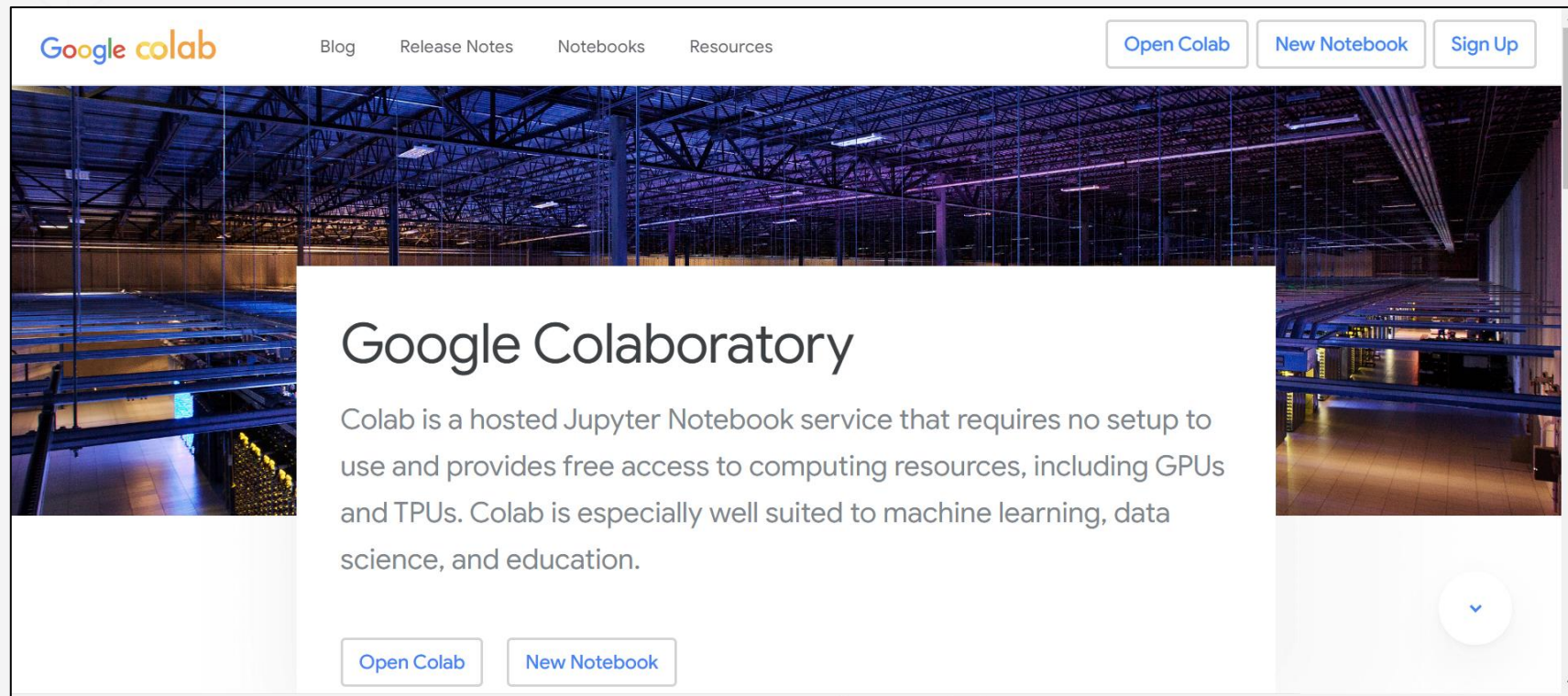
- ✓ Zero configuration required
- ✓ Free Cloud service with free GPU
- ✓ Write and execute code in Python

- ✓ Create/Upload/Share notebooks
- ✓ Import/Save notebooks from/to Google Drive
- ✓ Import/Publish notebooks from GitHub

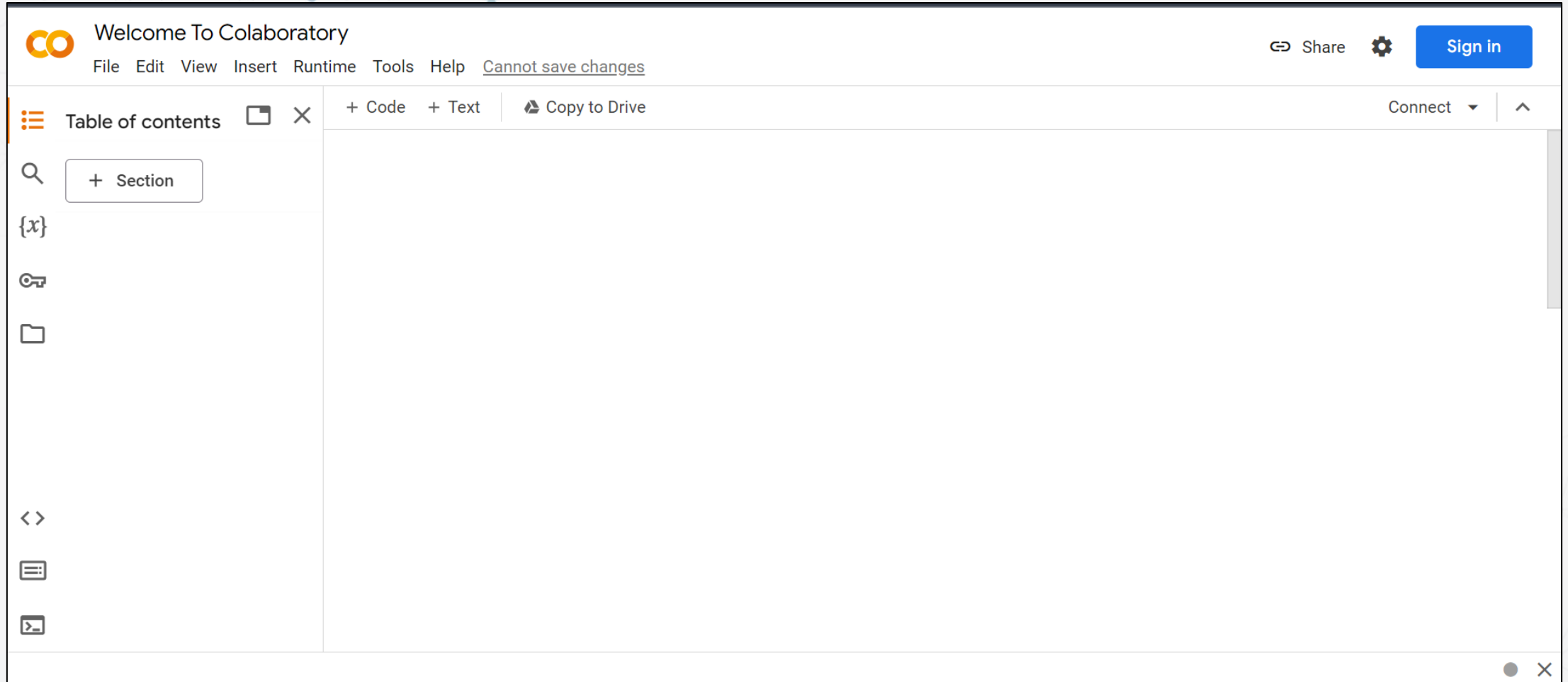
- ✓ Import external datasets e.g. from Kaggle
- ✓ Integrate PyTorch, TensorFlow, Keras, OpenCV

Create New Colab Notebook

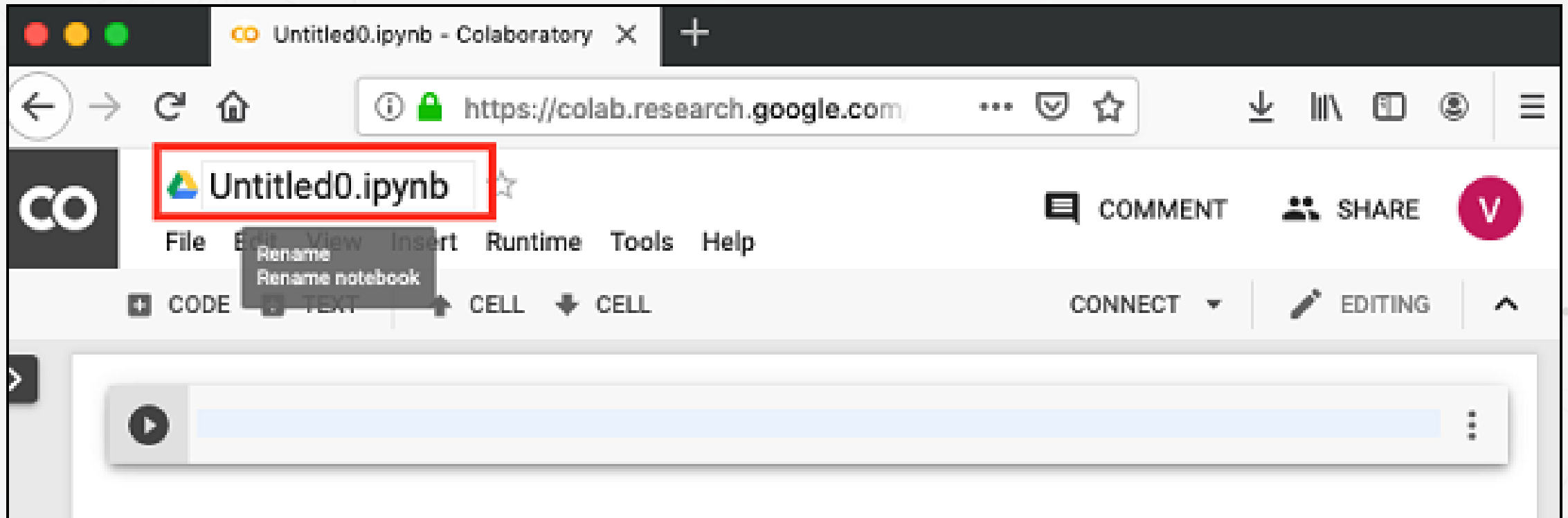
1. Open Colab @ <https://colab.google/>
2. New Notebook



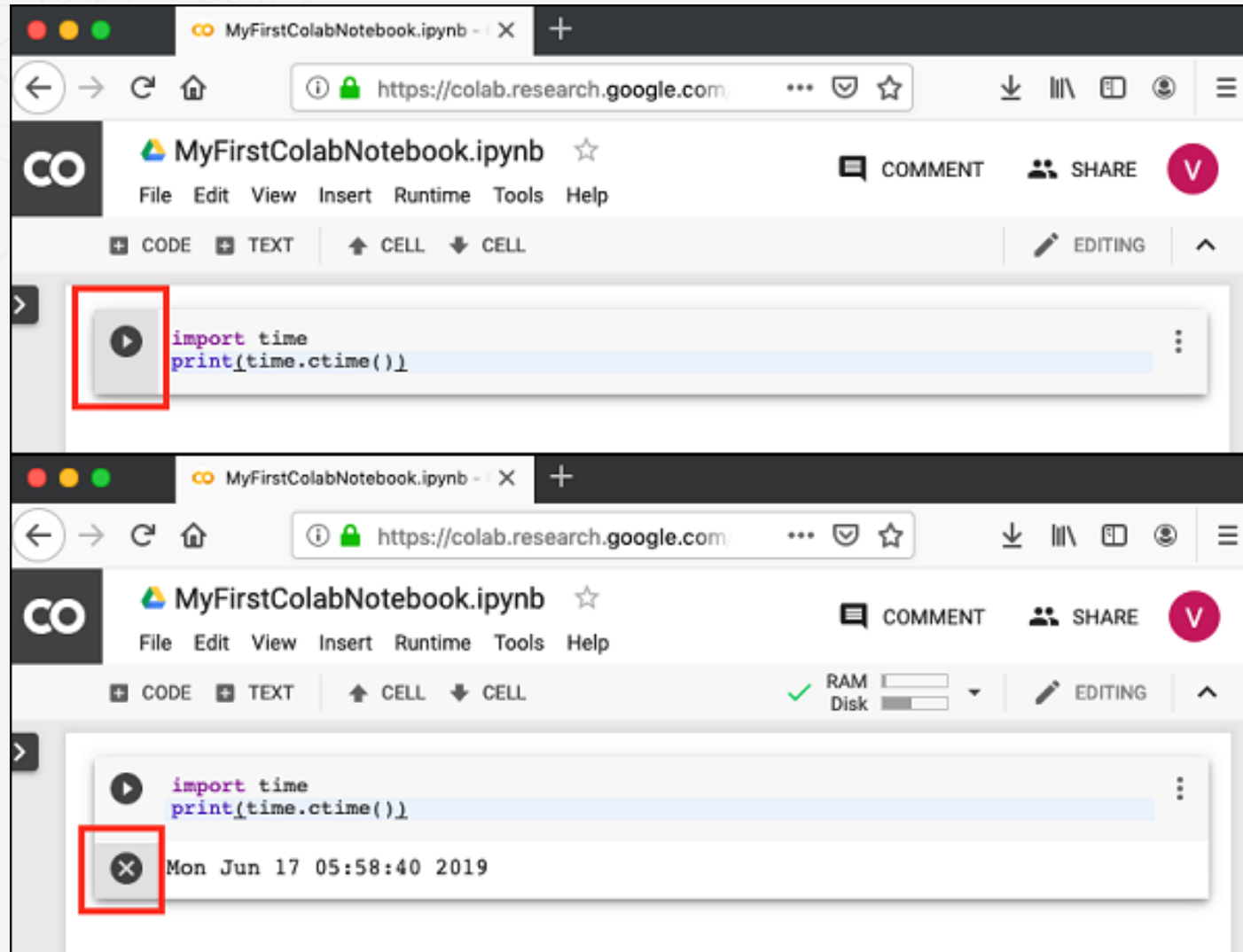
Create New Colab Notebook



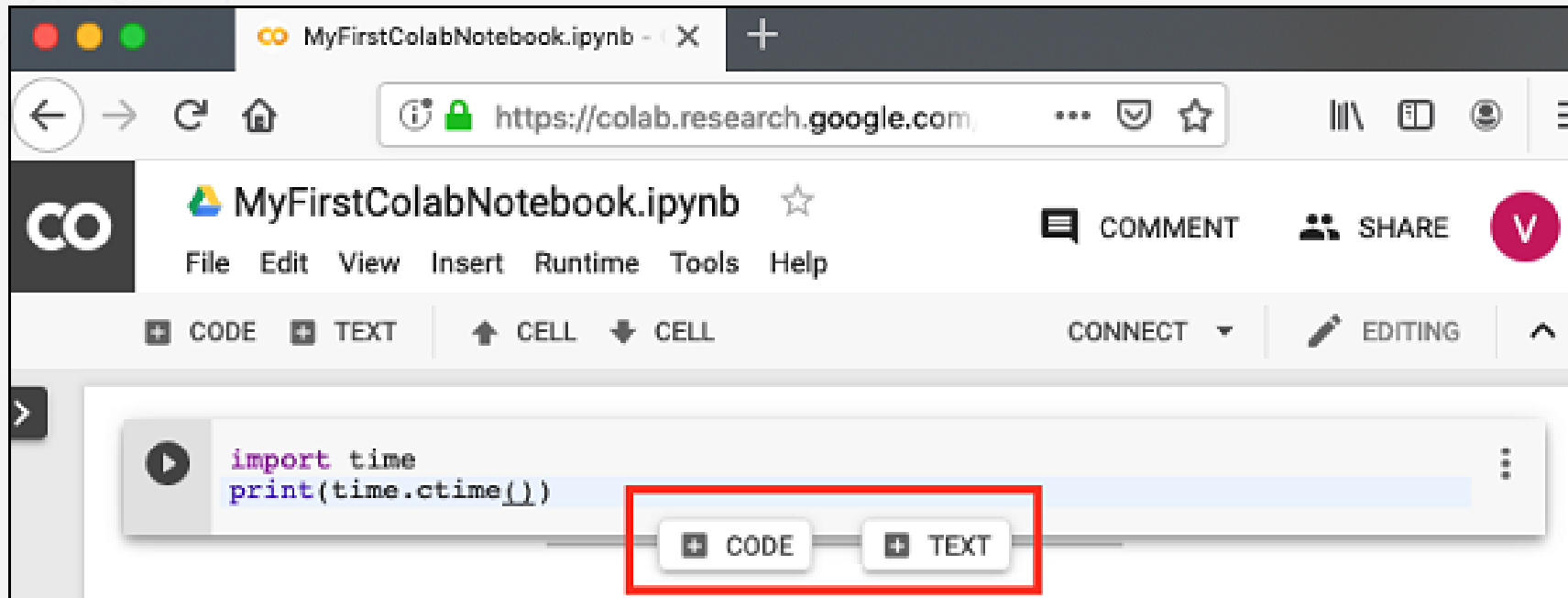
Set Notebook Name



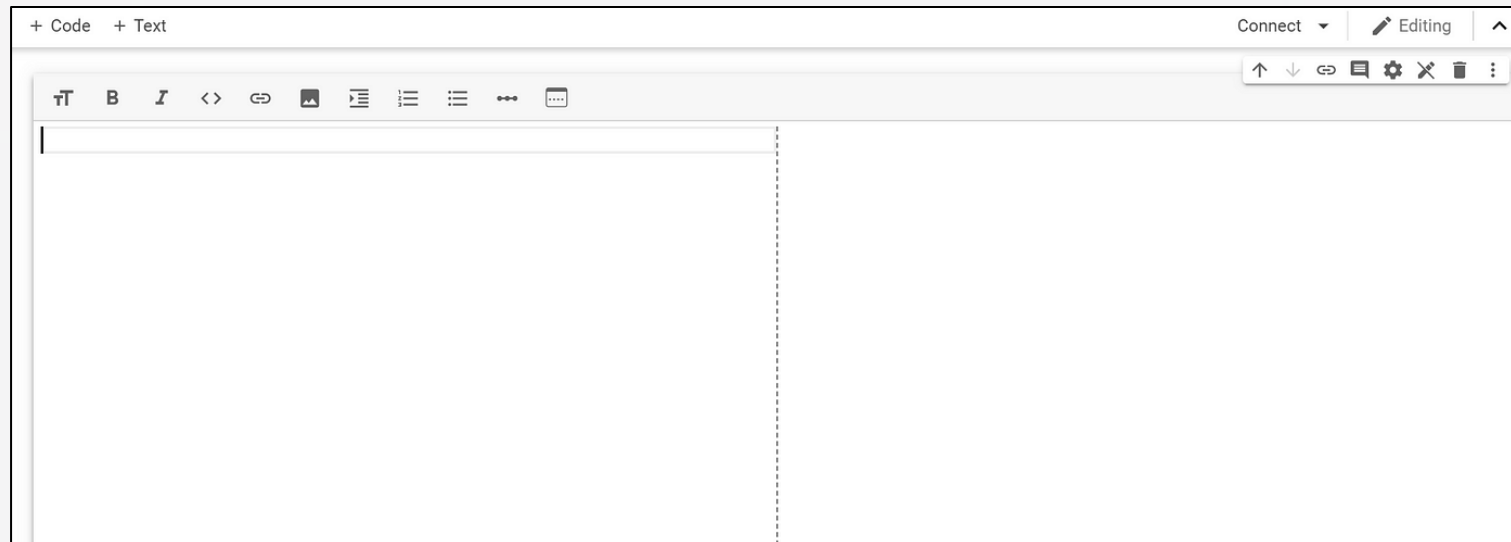
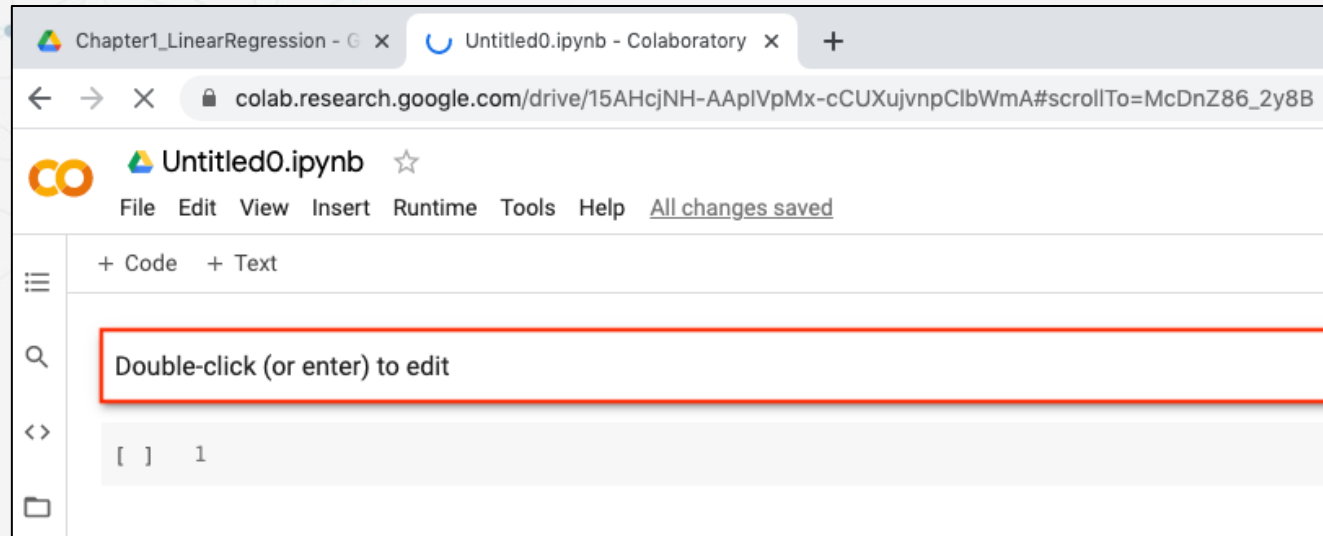
Add Code Cell



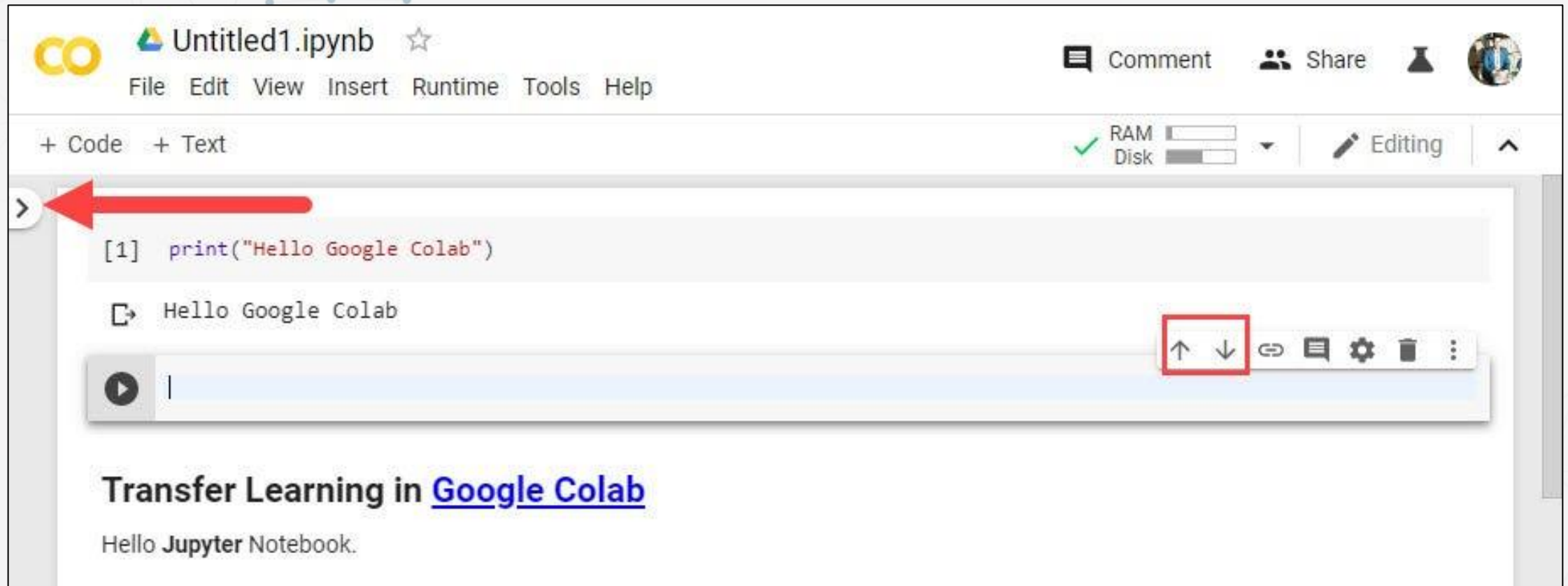
Add Code Cell



Add Text Cell



Change Cell Order & Delete Cell



The screenshot displays the Google Colab interface for a notebook titled "Untitled1.ipynb". The top menu bar includes "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help". On the right, there are buttons for "Comment", "Share", and a user profile icon. Below the menu, a toolbar shows "+ Code" and "+ Text" buttons, along with RAM and Disk usage indicators and an "Editing" mode button. The main workspace contains a single code cell. A red arrow points to the cell handle icon (a circle with a right-pointing chevron) on the left side of the cell. The cell contains the code `[1] print("Hello Google Colab")` and its output, "Hello Google Colab". Below the code input area, a toolbar contains icons for moving the cell up (up arrow), moving the cell down (down arrow), linking, commenting, settings, deleting (trash can), and a more options menu (three dots). The up and down arrow icons are highlighted with a red rectangular box. Below the code cell, the text "Transfer Learning in [Google Colab](#)" and "Hello Jupyter Notebook." are visible.

Upload File

The screenshot shows the Google Colab interface. On the left, the 'Files' sidebar is open, displaying a directory structure. A green box highlights the 'drive' folder, which contains a 'MyDrive' folder. Inside 'MyDrive', there are several subfolders ('Colab Notebooks', 'Daughter App', 'Other') and files ('Easter Trip.gmap', 'How to get started with Dr...', 'PWA workshop.gdoc', 'Salary_Data.csv', 'To learn and add.gsheets'). An orange arrow points from the 'Salary_Data.csv' file in the sidebar to the code cell on the right. The code cell contains two blocks of code. The first block mounts the drive, and the second block imports pandas and reads the CSV file. Below the code, the output shows the first five rows of the CSV data as a table.

Untitled0.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

Files

drive

MyDrive

- Colab Notebooks
- Daughter App
- Other
- Easter Trip.gmap
- How to get started with Dr...
- PWA workshop.gdoc
- Salary_Data.csv
- To learn and add.gsheets

sample_data

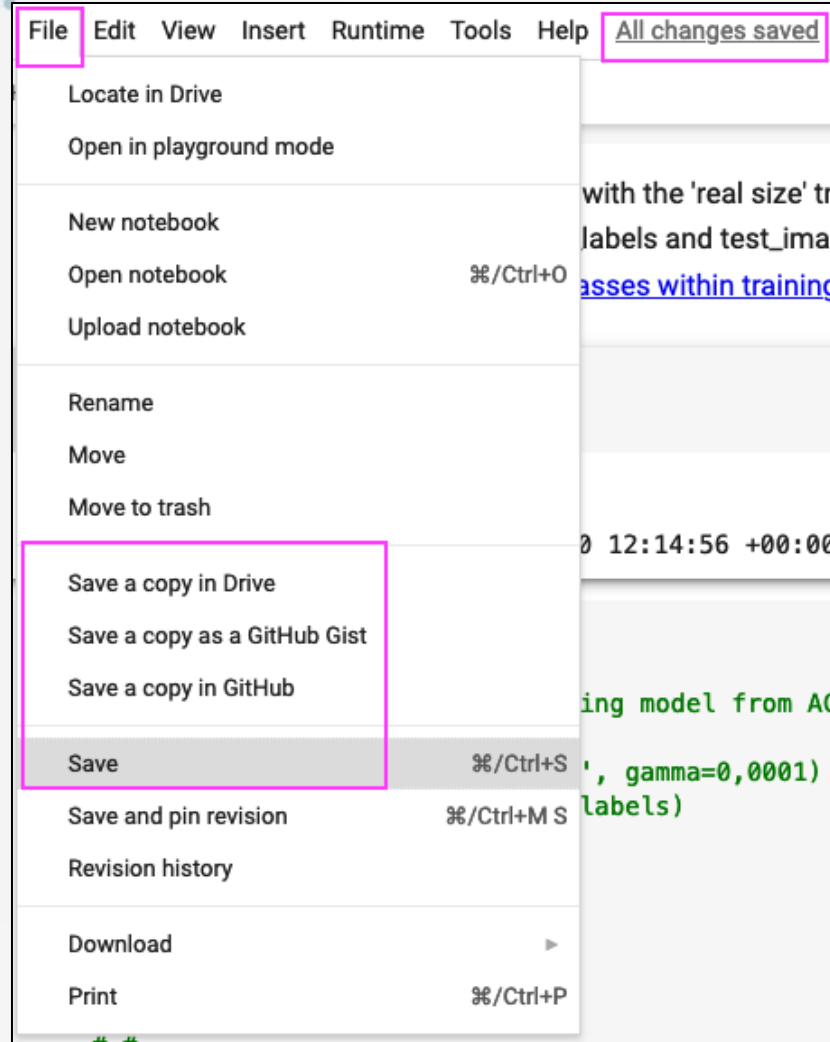
```
[1] from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive

[4] import pandas as pd
df = pd.read_csv('drive/MyDrive/Salary_Data.csv')
df.head()
```

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

File Tab



Pandas Library

Pandas is a software library written for the Python programming language for data manipulation and analysis.

It offers data structures and operations for manipulating numerical tables and time series.

Pandas excels in its ease of working with structured data formats such as tables, matrices, and time series data.



Sample Data

Sales data of each salesperson of a company over countries and their branches at different regions in each country.

Country	Region	Requester	Date of Purchase	Total	Quantity
India	North	John	2/7/2018 0:00:00	100,000.00	567
US	North	Bill	12/18/2017 0:00:00	120,000.00	3000
UK	North	Thomas	3/15/2016 0:00:00	140,000.00	345
Australia	East	John	1/18/2012 0:00:00	160,000.00	1000
Africa	East	Bill	12/6/2014 0:00:00	180,000.00	123
Singapore	East	Thomas	4/18/2018 0:00:00	200,000.00	1000
Mylasia	West	John	2/28/2010 0:00:00	1,000,000.00	7890
India	West	Bill	6/27/2016 0:00:00	240,000.00	200
US	West	Thomas	8/9/2011 0:00:00	26,000,000.00	1000
UK	North	John	5/14/2010 0:00:00	100,000.00	1000
Australia	North	Bill	3/13/2011 0:00:00	120,000.00	567
Africa	North	Thomas	1/31/2015 0:00:00	140,000.00	1000
Singapore	East	John	4/14/2013 0:00:00	160,000.00	892
Mylasia	East	Bill	2/8/2016 0:00:00	180,000.00	444
India	East	Thomas	10/13/2018 0:00:00	200,000.00	90
US	West	John	9/12/2011 0:00:00	220,000.00	90
UK	West	Bill	11/14/2017 0:00:00	240,000.00	90
Australia	West	Thomas	8/19/2011 0:00:00	260,000.00	90
Africa	North	John	4/22/2013 0:00:00	140,000.00	85
Singapore	North	Bill	3/12/2016 0:00:00	150,000.00	85

Load Data

- The loaded data is stored in a Pandas data structure called DataFrame.
- DataFrame's are usually referred by the variable name df
- df associated with Dataframe.

Load Data

From CSV File

```
import pandas  
df = pandas.read_csv("path_to_csv")
```

From Remote URL

```
import pandas  
df = pandas.read_csv("remote/url/path/pointing/to/csv")
```

Load Data

From DB

```
db = # Create DB connection object  
cur = db.cursor()  
cur.execute("SELECT * FROM <TABLE>")  
df = pd.DataFrame(cur.fetchall())
```

Understanding Data

1. shows you a gist of the data

```
df.head()
```

2. Some statistical information about your data

```
df.describe()
```

3. List of columns headers

```
df.columns.values
```

A gist of the data

```
In [6]: df.head()
```

```
Out[6]:
```

	Country	Region	Requester	Date of Purchase	Total	Quantity
0	India	North	John	9/16/2016 0:00:00	100000	567
1	US	North	Bill	10/19/2018 0:00:00	120000	3000
2	UK	North	Thomas	6/10/2014 0:00:00	140000	345
3	Australia	East	John	11/23/2010 0:00:00	160000	1000
4	Africa	East	Bill	2/17/2010 0:00:00	180000	123

Know your columns

```
In [7]: df.columns.values
```

```
Out[7]: array(['Country', 'Region', 'Requester', 'Date of Purchase', 'Total',  
              'Quantity'], dtype=object)
```

```
In [8]: df.describe()
```

```
Out[8]:
```

	Total	Quantity
count	2.000000e+01	20.000000
mean	1.502500e+06	977.900000
std	5.769280e+06	1761.923497
min	1.000000e+05	85.000000
25%	1.400000e+05	90.000000
50%	1.700000e+05	505.500000
75%	2.250000e+05	1000.000000
max	2.600000e+07	7890.000000

Select Columns

```
# 1. Create a list of columns to be selected
columns_to_be_selected = ["Total", "Quantity", "Country"]

# 2. Use it as an index to the DataFrame
df[columns_to_be_selected]

# 3. Using loc method
df.loc[columns_to_be_selected]
```

	Total	Quantity	Country
0	100000	567	India
1	120000	3000	US
2	140000	345	UK
3	160000	1000	Australia
4	180000	123	Africa
5	200000	1000	Singapore
6	1000000	7890	Mylasia
7	240000	200	India
8	26000000	1000	US
9	100000	1000	UK
10	120000	567	Australia

Select Rows

DataFrame provides numerical indexing (0, 1, 2...) by default.

```
# 1. using numerical indexes - iloc
df.iloc[0:3, :]

# 2. using labels as index - loc
row_index_to_select = [0, 1, 4, 5]
df.loc[row_index_to_select]
```

```
In [38]: df.loc[[0, 1, 4, 5]]
```

```
Out[38]:
```

	Country	Region	Requester	Date of Purchase	Total	Quantity
0	India	North	John	2016-09-16	100000	567
1	US	North	Bill	2018-10-19	120000	3000
4	Africa	East	Bill	2010-02-17	180000	123
5	Singapore	East	Thomas	2017-08-14	200000	1000

Filter Data

Filter out the rows that satisfy a certain condition.

1. Total sales > 200000

```
df[df["Total"] > 200000]
```

2. Total sales > 200000 and in UK

```
df[(df["Total"] > 200000) & (df["Country"] == "UK")]
```

scikit-learn Library

scikit-learn is a free software machine learning library for the Python programming language.

It features various classification, regression and clustering algorithms such as including support-vector machines, random forests, gradient boosting, k-means.



scikit-learn Library Applications

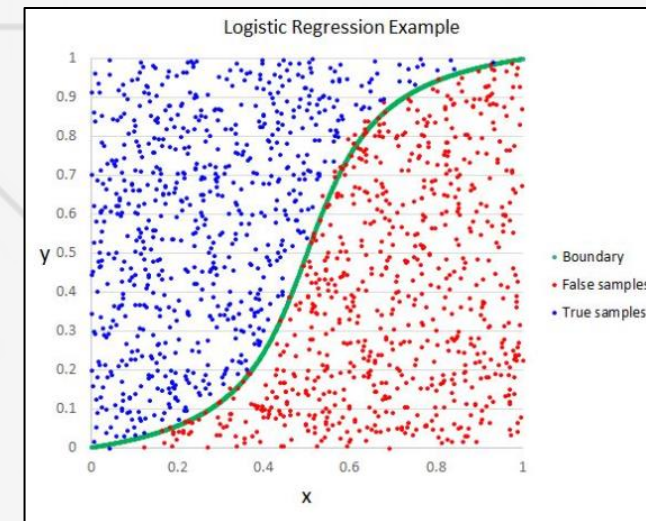
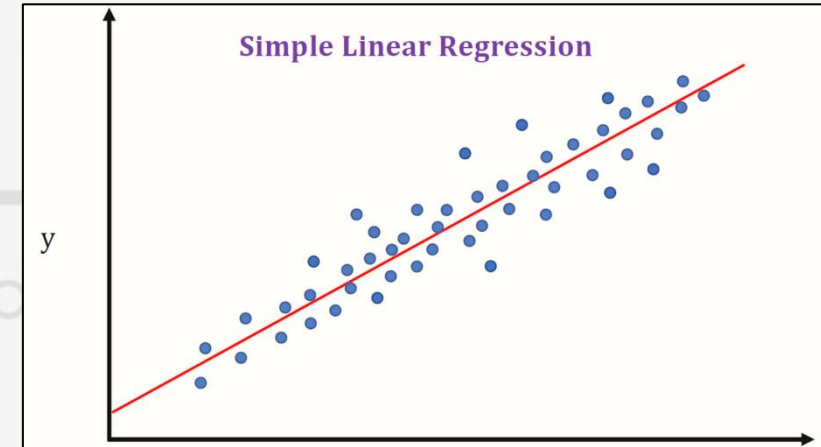
Scikit-Learn can be used to implement a wide variety of AI models, from both supervised and unsupervised learning.

Applications can be divided into the following groups:

1. Classification
2. Regressions
3. Dimension reduction
4. Data preprocessing and visualization

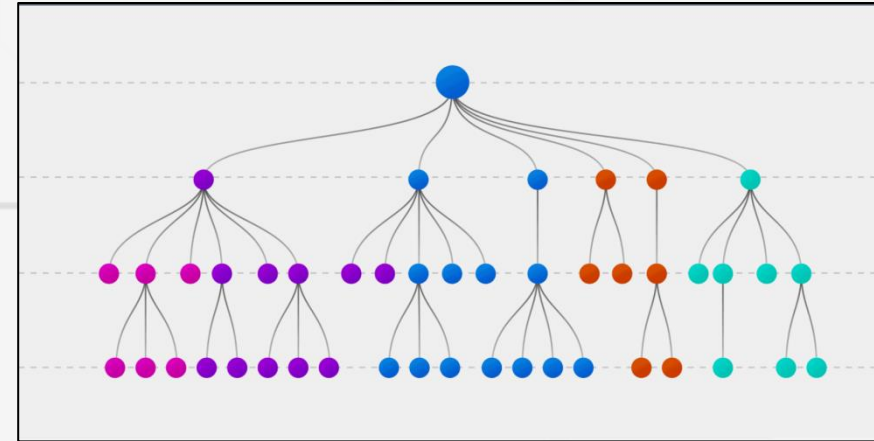
Supported Supervised Learning Algorithms

1. **Linear regression:** A model that fits a linear equation to the data.
2. **Logistic regression:** A model that uses a logistic function to estimate the probability of a binary outcome.

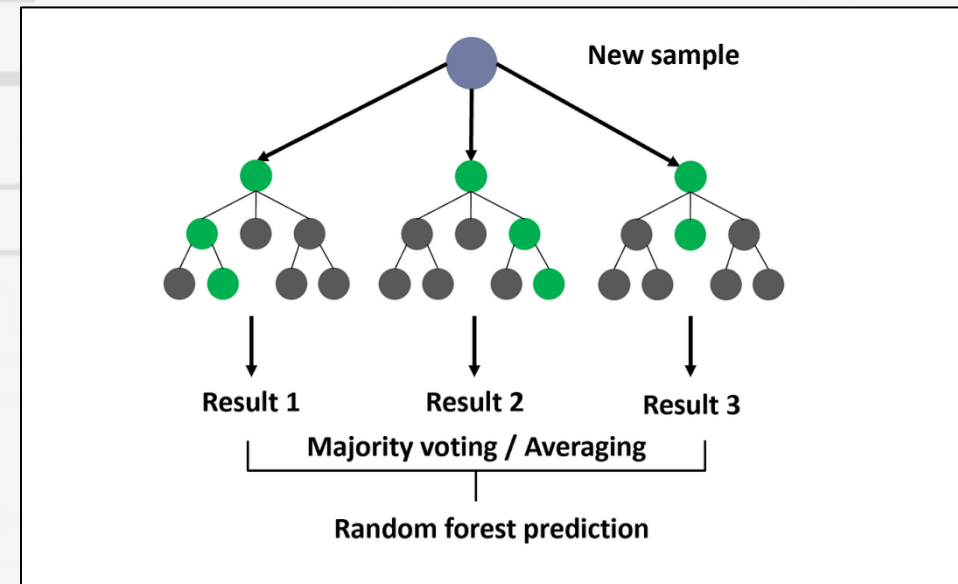


Supported Supervised Learning Algorithms

3. Decision trees: A model that recursively partitions the data into subsets based on the most discriminative features.



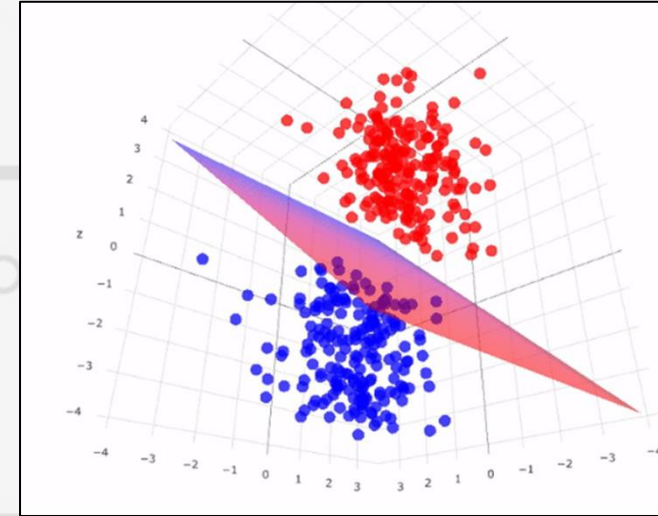
4. Random forests: An ensemble model that combines multiple decision trees to reduce overfitting.



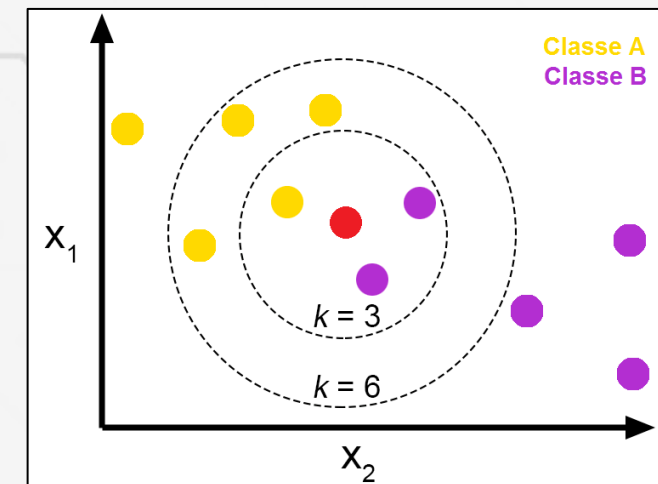
Supported Supervised Learning Algorithms

5. Support vector machines (SVM):

A model that finds the hyperplane that best separates the data into different classes.

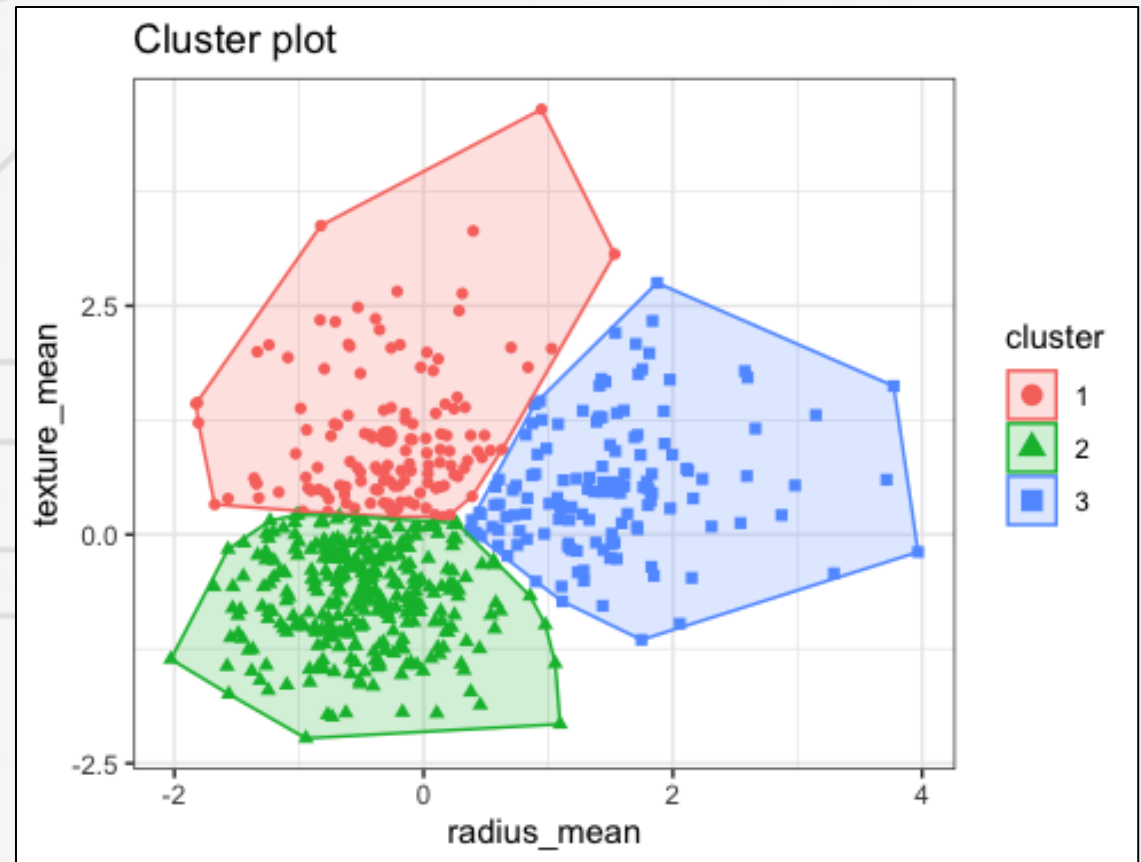


6. k-Nearest Neighbors (k-NN): A model that predicts the class of a sample based on the classes of its k nearest neighbors in the training data.



Supported Unsupervised Learning Algorithms

Clustering: which involves grouping similar instances together based on a given distance metric.



Advantages of scikit-learn Library

- ✓ **Simplified application of machine learning tools, data analytics, and data visualization**
- ✓ **Commercial use without licensing fees**
- ✓ **High degree of flexibility in fine-tuning models**
- ✓ **Usable in different contexts**

scikit-learn Library in Python

```
# Import Modules
from sklearn.datasets import load_iris
from sklearn import tree

# Load Iris Dataset
iris = load_iris()

# Define X and Y Variable
X, y = iris.data, iris.target

# Set up the Decision Tree Classifier
clf = tree.DecisionTreeClassifier()

# Train it on the Iris Data
clf = clf.fit(X, y)
```